

ASSIGNMENT 1

The UMBC campus currently faces significant challenges with parking management. The manual enforcement of parking regulations leads to unauthorized parking, inefficiencies in finding available spots, and frustration among commuters and visitors. The purpose of this project is to design and implement a robust, automated Database Management System (DBMS) in PostgreSQL to handle UMBC's parking infrastructure.

This system will track parking inventory, manage permit issuance, and automate operations in real-time based on sensor data. The primary Parking Users interacting with this system include:

- Students: Commuters and residents who register vehicles and hold student permits.
- Faculty: University staff who hold premium faculty permits.
- Visitors: Guests who book temporary parking reservations for specific time windows.
- Parking Admin: Administrators who manage parking inventory, define rules, and review financial transactions.

System Scope The database will fully support the lifecycle of parking permits (apply, issue, expire), manage parking inventory (lots, zones, and spots), and process visitor reservations with precise time windows. Furthermore, it will record arrival and departure events triggered by ground sensors to dynamically update spot occupancy. Using this sensor data, the system will automatically generate tickets for unauthorized parking and track ticket payment statuses. Out of scope for this database implementation are interactive maps, real physical sensor hardware integration, full mobile applications, and external payment gateway processing

Use-Cases

1. A Student registers a vehicle and purchases a commuter permit.
2. A Faculty member registers a vehicle and is issued a faculty permit for restricted lots.
3. A Visitor books a parking reservation for a specific time window and receives confirmation.
4. A Sensor detects a vehicle arrival and updates the specific parking spot's status to "Occupied".
5. A Sensor detects a vehicle departure and updates the specific parking spot's status to "Available".
6. The system automatically cross-references an occupied spot against active permits and current reservations to verify authorization.
7. The system automatically generates a Ticket when an unauthorized vehicle parks in a faculty or reserved spot.
8. A Parking Admin adds a new parking lot to the database and defines its individual spots and zones.
9. A Student or Faculty member views their currently active permits and any outstanding tickets.
10. A user submits a payment for a parking ticket, updating the ticket status from "Issued" to "Paid".
11. A Parking Admin generates a report showing the current availability and occupancy rates of all campus lots.
12. The system prevents a double-booking anomaly when two visitors attempt to reserve the same spot for the exact same time window.

Assignment 2

System module	Student	Faculty	Visitor	Admin
Permits	Apply/view own	Apply/view own	No access	Manage All/ Define types

Inventory	View availability	View availability	View availability	Create/ Edit / Delete
Reservations	No access	No access	Create/Pay/View	View All
Ticketing	View/ Pay own	View/ Pay own	View/ Pay own	Auto-Generate/ review

The system will enforce the following 10 business rules to maintain data integrity:

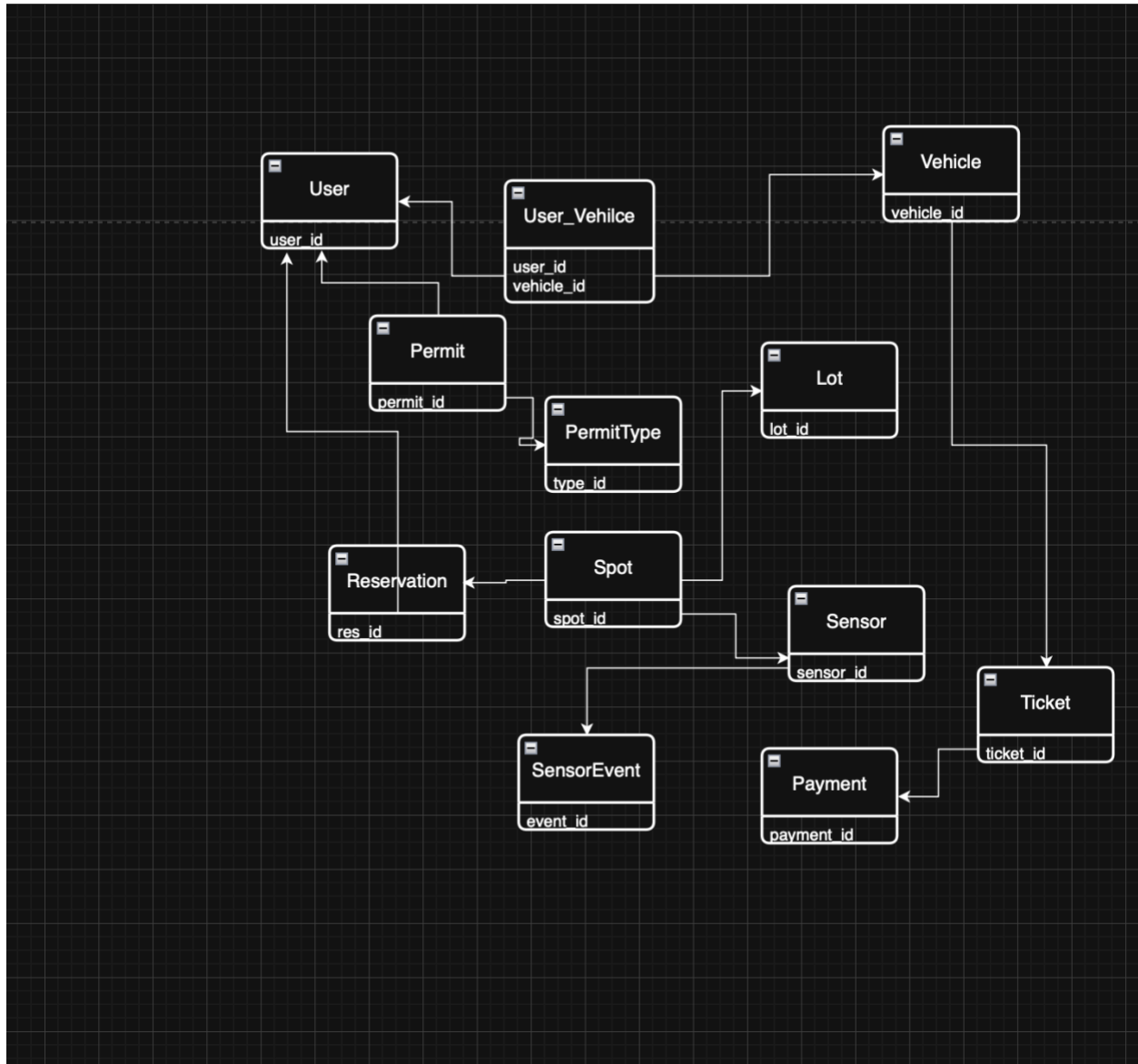
1. (A) Table Constraint: A vehicle's license plate must be entirely unique within the system (Unique constraint).
2. (A) Table Constraint: A specific parking spot can only belong to one designated parking lot (Foreign Key)
3. (A) Table Constraint: A visitor reservation must have a valid timeframe where the end time is strictly greater than the start time. (Check constraint)
4. (A) Table Constraint: Ticket fine amounts must be strictly greater than \$0.00 (Check constraint)
5. (A) Table Constraint: A user ID and vehicle ID pairing in the registration table must be unique (Composite Primary Key)
6. (B) Trigger/Function: A vehicle cannot hold more than one actively valid permit at the exact same time.
7. (B) Trigger/Function: If a sensor detects an arrival event, the corresponding spot's occupancy status automatically updates to occupied.
8. (B) Trigger/Function: The auto-ticketing system will generate a ticket if a vehicle with a 'Student' permit parks in a spot designated strictly for faculty.
9. (B) Trigger/Function: A parking ticket cannot be paid twice; once the status is paid, further payment attempts are rejected.
10. (B) Trigger/Function: The system prevents double-booking; if a time window overlaps with an existing confirmed reservation for a specific spot, the new reservation is rejected.

The Scenario: Two visitors (User A and User B) simultaneously attempt to book a reservation for "Visitor Lot 1, Spot 5" for the exact same time block (Tuesday, 10:00 AM - 12:00 PM). The Outcome: The database will utilize transaction locking to prevent a double-booking anomaly. User A's transaction will acquire a lock on the spot and successfully commit the reservation. User B's transaction, running milliseconds behind, will be forced to wait. Once User A's transaction completes, User B's transaction will evaluate the new state, see the time conflict constraint, and fail, returning an "Unavailable" error to User B.

To maintain a flexible and robust architecture, PermitType (e.g., Student, Faculty, ADA) was modeled as an independent entity separate from the Permit issuance entity. This normalizes the database and allows Parking Administrators to update the pricing or rules of a specific permit tier without retroactively altering historical permit records.

Additionally, the relationship between Users and Vehicles is modeled as a Many-to-Many (M:N) relationship using a User_Vehicle associative entity. This reflects the real-world scenario where a single user may register multiple cars, and a single car might be shared and registered by multiple users (e.g., siblings attending the same university). Finally, time windows for reservations are explicitly modeled using start_time and end_time timestamp attributes. This facilitates highly efficient overlapping time

queries, which are critical for preventing concurrency anomalies and double-booking during high-traffic campus events.



Assignment 3

The relational schema list:

- Users (user_id, name, email, role_id → Roles(role_id))
- Roles (role_id, role_name)
- Vehicles (vehicle_id, license_plate, make, model, color)
- User_Vehicles (user_id → Users(user_id), vehicle_id → Vehicles(vehicle_id))
- PermitTypes (type_id, type_name, cost, duration_days)

- Permits (permit_id, issue_date, expiry_date, vehicle_id
→ Vehicles(vehicle_id), type_id → PermitTypes(type_id))
- Lots (lot_id, lot_name, location)
- Spots (spot_id, spot_number, spot_type, is_occupied, lot_id → Lots(lot_id))
- Reservations (res_id, start_time, end_time,
status, user_id → Users(user_id), spot_id → Spots(spot_id))
- Sensors (sensor_id, sensor_model, spot_id → Spots(spot_id))
- SensorEvents (event_id, event_type, event_timestamp, sensor_id → Sensors(sensor_id))
- Tickets (ticket_id, issue_timestamp, fine_amount,
status, vehicle_id → Vehicles(vehicle_id), spot_id → Spots(spot_id))
- Payments (payment_id, amount, payment_timestamp, ticket_id → Tickets(ticket_id))

Normalizations:

for the spots table: spot_id → {spot_number, lot_id}

If we kept the lot_name inside the spots table, changing a lot's name would require updating hundreds of rows, this is called an update anomaly. By moving lot_name to a separate Lots table we satisfy the Third Normal Form.

For the permits table: permit_id → {type_id, issue_date}

If we stored the permit cost directly in the permits table deleting the last student permit would also delete the knowledge of how much a student permit cost. This is a deletion anomaly. Separating PermitTypes into its own table ensures the database is in BCNF.

Candidate keys

Vehicles: the license_plate is a candidate key because it is unique to every car

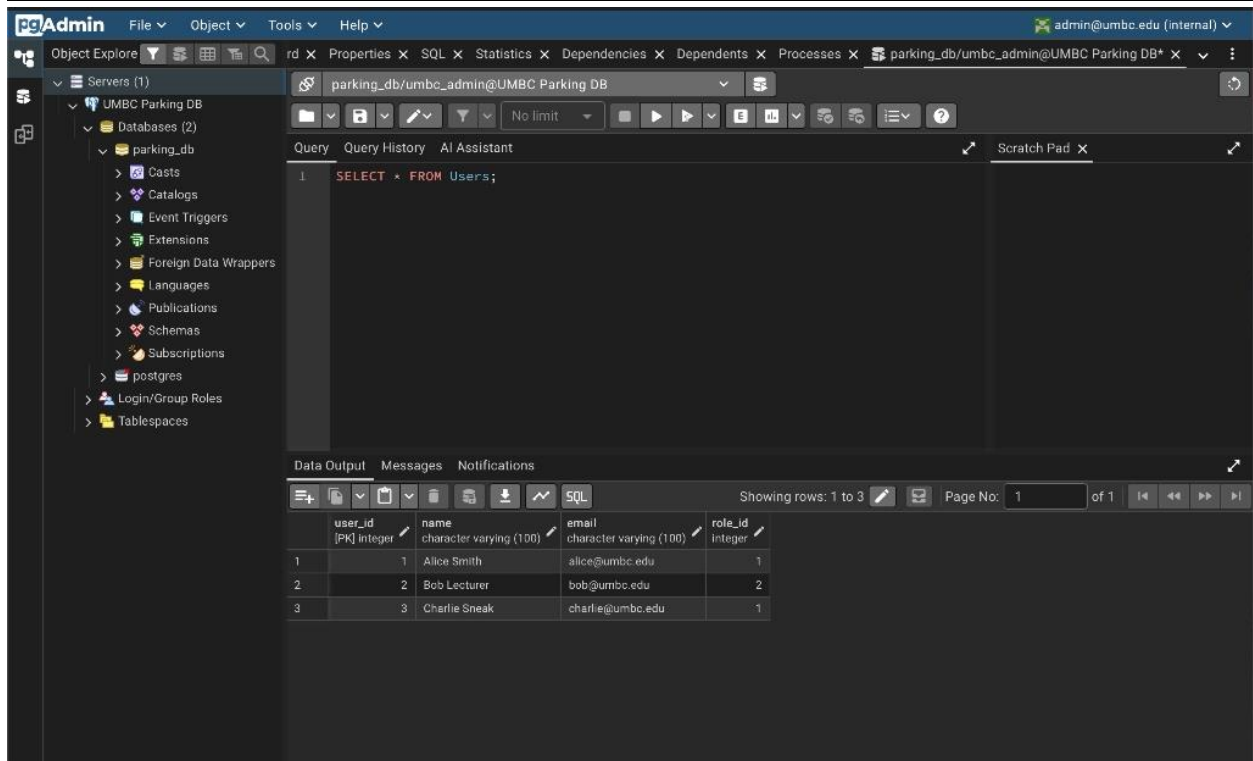
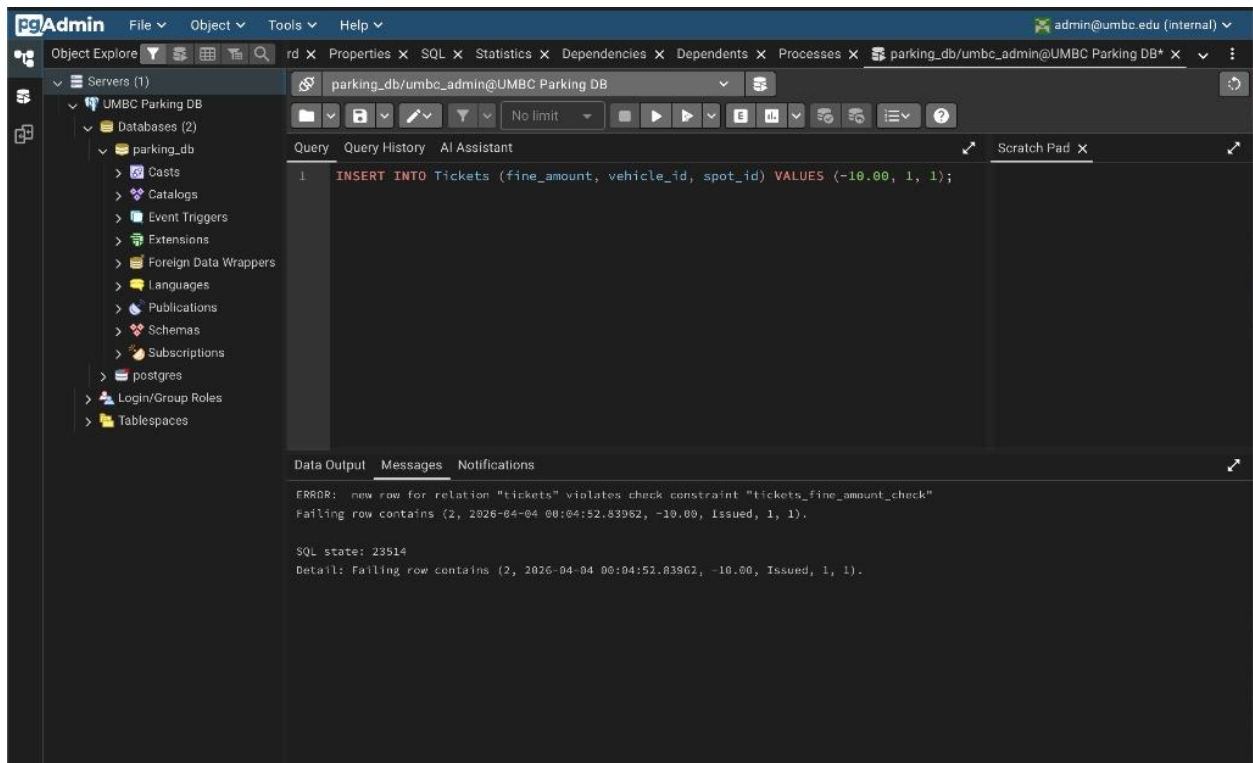
Composite keys

User_Vehicles: uses a composite primary key (user_id, vehicle_id). This allows a single user to register multiple cars and a single car to be shared by multiple users while preventing duplicate entries for the same pair.

Naming conventions:

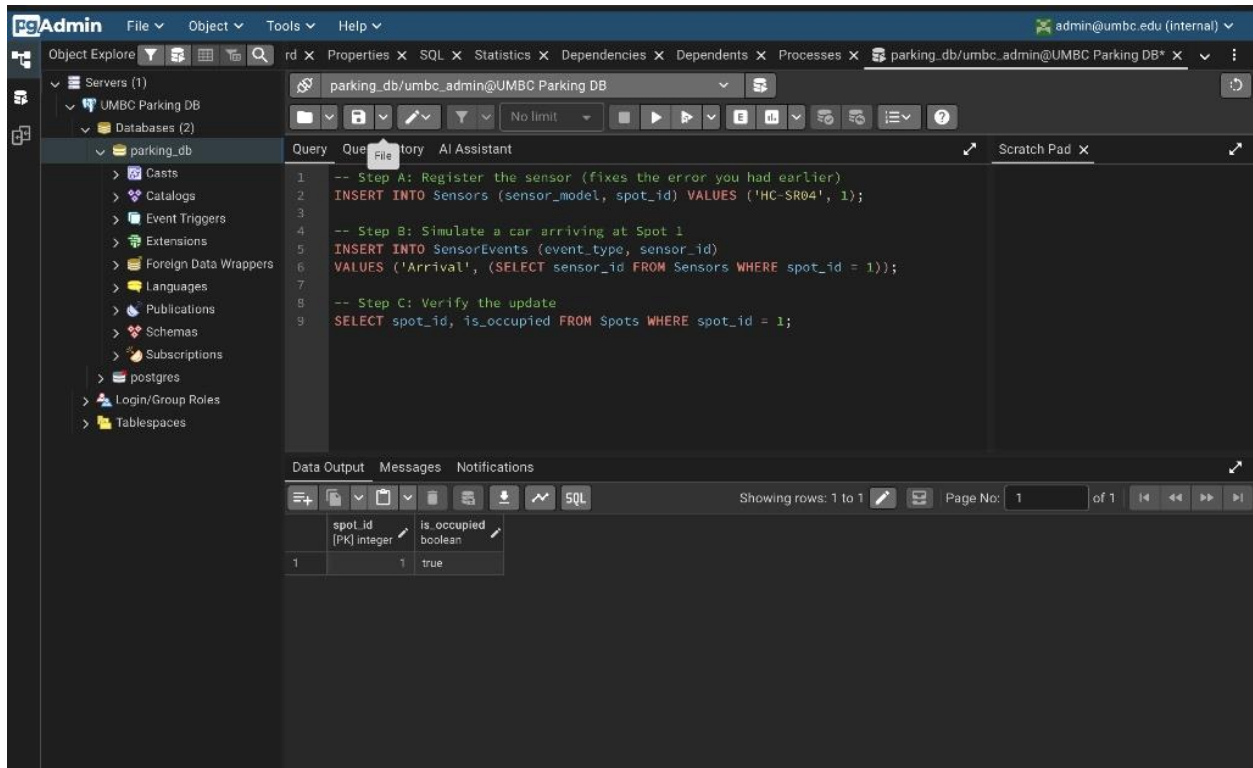
- Table Names: PascalCase
- Column Names: snake_case
- ID columns: consistent use of table_name_id for primary keys
- Timestamps: use of the TIMESTAMP data type for all time-sensitive fields.

Assignment 4

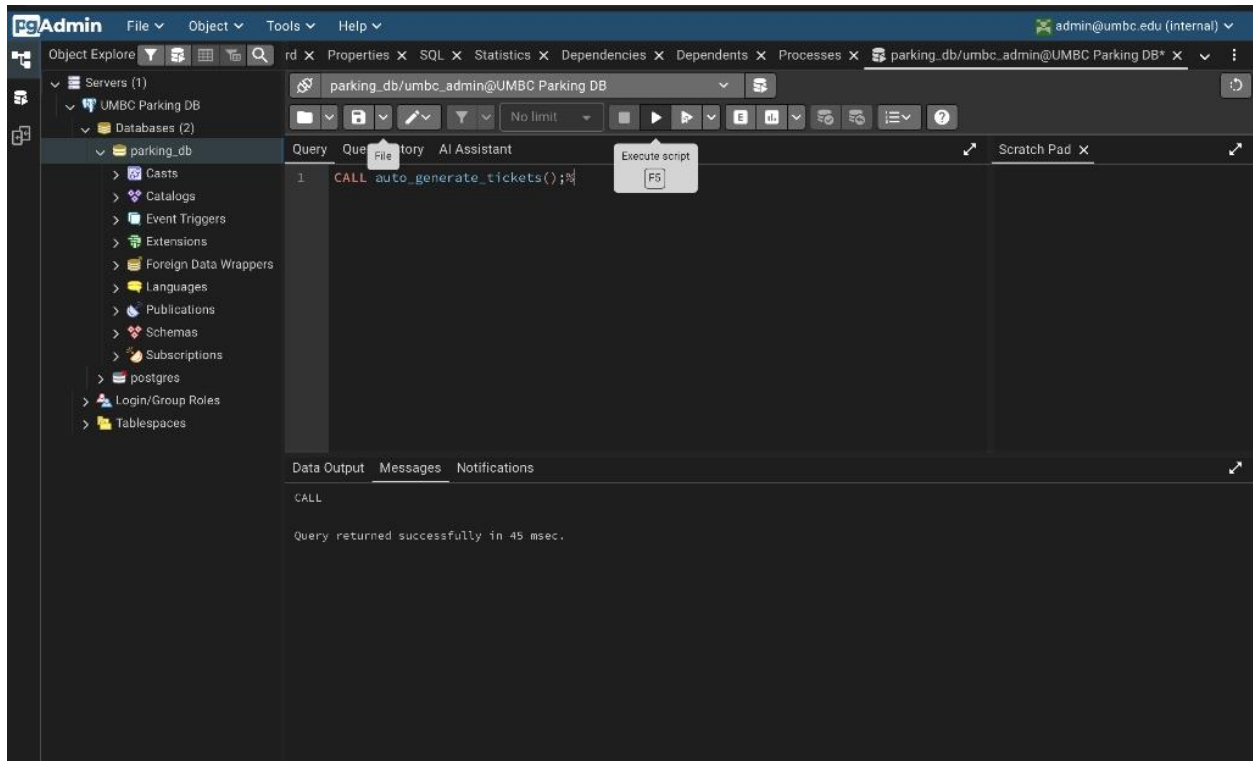


Assignment 5

Trigger Test: Screenshot showing is_occupied flipping from false to true after an 'Arrival' event.

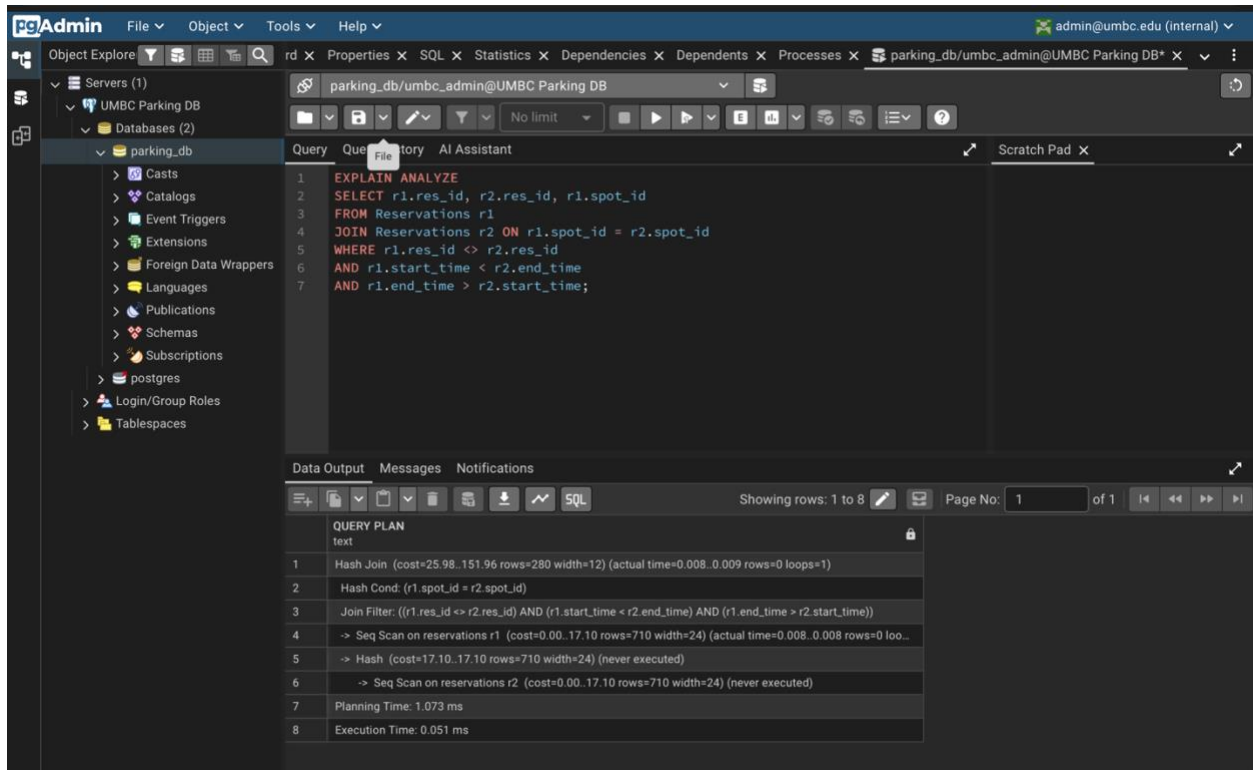


Procedure Test: Screenshot of the Tickets table showing new violations generated by CALL auto_generate_tickets();



Performance Evidence:

Before



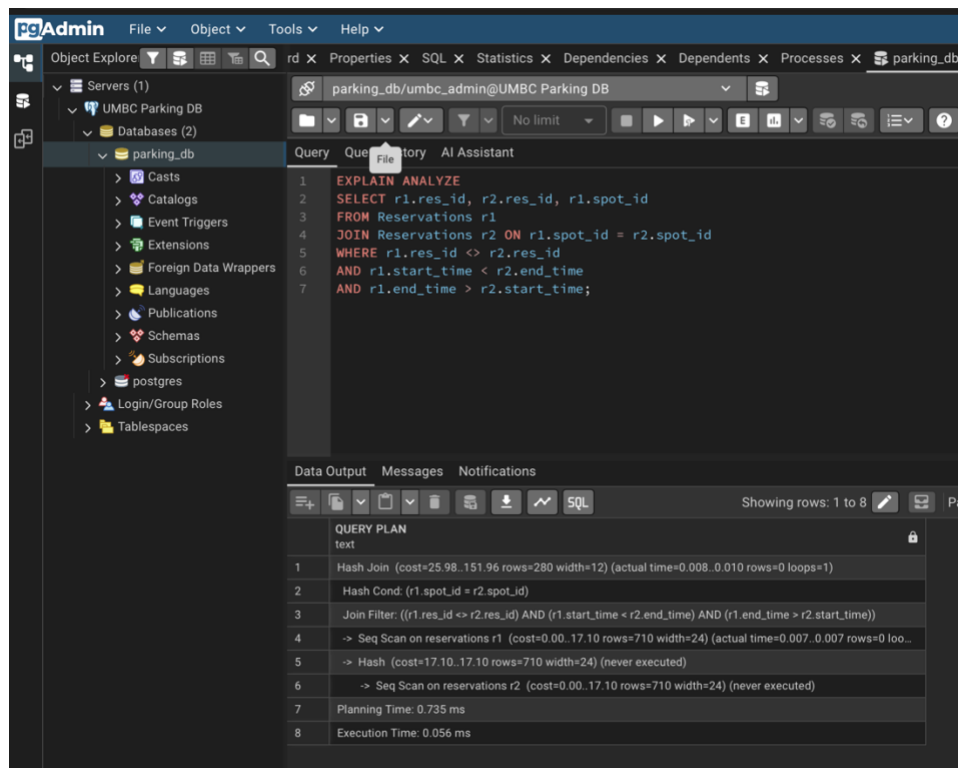
The screenshot shows the pgAdmin interface with a query window open. The query is an `EXPLAIN ANALYZE` statement for a join operation. The query plan output is visible at the bottom.

```
1 EXPLAIN ANALYZE
2 SELECT r1.res_id, r2.res_id, r1.spot_id
3 FROM Reservations r1
4 JOIN Reservations r2 ON r1.spot_id = r2.spot_id
5 WHERE r1.res_id <> r2.res_id
6 AND r1.start_time < r2.end_time
7 AND r1.end_time > r2.start_time;
```

The query plan output shows the following details:

Step	Operation	Cost	Rows	Width	Actual Time	Loops
1	Hash Join	25.98..151.96	rows=280	width=12	(actual time=0.008..0.009	rows=0 loops=1)
2	Hash Cond				(r1.spot_id = r2.spot_id)	
3	Join Filter				((r1.res_id <> r2.res_id) AND (r1.start_time < r2.end_time) AND (r1.end_time > r2.start_time))	
4	Seq Scan on reservations r1	cost=0.00..17.10	rows=710	width=24	(actual time=0.008..0.008	rows=0 loops=1)
5	Hash	cost=17.10..17.10	rows=710	width=24	(never executed)	
6	Seq Scan on reservations r2	cost=0.00..17.10	rows=710	width=24	(never executed)	
7	Planning Time				1.073 ms	
8	Execution Time				0.051 ms	

After



Assignment 6

System Walkthrough:

Permit management: The system utilizes the `issue_permit` function to grant access to students and faculty while enforcing business rules that prevent a single vehicle from holding multiple active permits simultaneously.

Real-Time Occupancy: Ground sensors automate the tracking of parking inventory. As demonstrated in the system tests inserting an arrival event into the `SensorEvents` table triggers an immediate update to the `Spots` table, flipping the `is_occupied` to true.

Automated Enforcement: The `auto_generate_tickets()` procedure provides a hands off approach to enforcement by scanning all occupied spots and automatically issuing a 50 dollar fine to vehicles without authorization.

Revenue Tracking: The system closes the operational loop by allowing payments to be recorded against specific `ticket_id` records ensuring all financial transactions are audited.

Concurrency control:

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

Showing rows: 1 to 1✎📄Page No: 1

	issue_permit void🔒
1	

Total rows: 1

Query complete 00:00:00.049

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

Showing rows: 1 to 2✎📄Page No: 1

	permit_id [PK] integer✎	issue_date date✎	expiry_date date✎	vehicle_id integer✎	type_id integer✎
1	1	2023-01-01	2023-06-01	1	1
2	2	2026-05-01	2026-10-28	1	1

Total rows: 2

Query complete 00:00:00.159

✔ Successfully run. Total query runtime:

Data OutputMessagesNotifications

WARNING: there is already a transaction in progress

Total rows:

Waiting for the query to complete... 00:00:16.717

UMBC Parking Management System

[ard](#) [Lots & Sensors](#) [Permits](#) [Reservations](#) [Enforcement](#) [Payments](#) [SQL Reports](#) [Concurrency Demo](#)

Concurrency Control Demo

Session A (User 1)

Run Session A (Should Succeed)

Session B (User 2 - Overlap)

Run Session B (Should Fail)

Session B succeeded (Run A first!)

	res_id	name	lot_name	spot_number	start_time	end_time	status
0	33	Bob Lecturer	Lot 4	12	2026-06-01 10:00:00	2026-06-01 12:00:00	Confirmed
1	29	Bob Lecturer	Lot 4	12	2026-06-01 10:00:00	2026-06-01 12:00:00	Confirmed
2	35	Bob Lecturer	Lot 4	12	2026-06-01 10:00:00	2026-06-01 12:00:00	Confirmed
3	31	Bob Lecturer	Lot 4	12	2026-06-01 10:00:00	2026-06-01 12:00:00	Confirmed
4	27	Bob Lecturer	Lot 4	12	2026-06-01 10:00:00	2026-06-01 12:00:00	Confirmed
5	34	Alice Smith	Lot 4	12	2026-06-01 09:00:00	2026-06-01 11:00:00	Confirmed
6	26	Alice Smith	Lot 4	12	2026-06-01 09:00:00	2026-06-01 11:00:00	Confirmed
7	28	Alice Smith	Lot 4	12	2026-06-01 09:00:00	2026-06-01 11:00:00	Confirmed
8	32	Alice Smith	Lot 4	12	2026-06-01 09:00:00	2026-06-01 11:00:00	Confirmed
9	30	Alice Smith	Lot 4	12	2026-06-01 09:00:00	2026-06-01 11:00:00	Confirmed
10	23	Bob Lecturer	Lot 4	12	2026-05-01 11:00:00	2026-05-01 13:00:00	Confirmed
11	2	Bob Lecturer	Lot 4	12	2026-05-01 11:00:00	2026-05-01 13:00:00	Confirmed
12	4	Bob Lecturer	Lot 4	12	2026-05-01 11:00:00	2026-05-01 13:00:00	Confirmed
13	16	Bob Lecturer	Lot 4	12	2026-05-01 11:00:00	2026-05-01 13:00:00	Confirmed
14	18	Bob Lecturer	Lot 4	12	2026-05-01 11:00:00	2026-05-01 13:00:00	Confirmed
15	24	Bob Lecturer	Lot 4	12	2026-05-01 11:00:00	2026-05-01 13:00:00	Confirmed
16	1	Alice Smith	Lot 4	12	2026-05-01 10:00:00	2026-05-01 12:00:00	Confirmed
17	22	Alice Smith	Lot 4	12	2026-05-01 10:00:00	2026-05-01 12:00:00	Confirmed
18	17	Alice Smith	Lot 4	12	2026-05-01 10:00:00	2026-05-01 12:00:00	Confirmed
19	15	Alice Smith	Lot 4	12	2026-05-01 10:00:00	2026-05-01 12:00:00	Confirmed
20	8	Alice Smith	Lot 4	12	2026-05-01 10:00:00	2026-05-01 12:00:00	Confirmed
21	5	Alice Smith	Lot 4	12	2026-05-01 10:00:00	2026-05-01 12:00:00	Confirmed

Permit Management

Issue Permit

Active Permits

Vehicle

GUEST-11

Permit Type

Premium Faculty

Start Date

2026/05/07

Issue Permit

	name	license_plate	expiry_date
empty			

Transaction Failed: Vehicle 3 already has an active permit. CONTEXT: PL/pgSQL function issue_permit(integer,integer,date) line 5 at RAISE